

# Meeting 10/03/2026

## Summary

### Action Items & Next Steps

---

- Implement unit testing for React components; integrate into CI on merge pipeline
- Set up Docker networking: bridges, inter-container communication, and node port mapping
- Explore Docker Scout CVE scanning commands and integrate into the pipeline
- Configure Docker volumes and understand persistent storage for containers
- Set up LocalStack and perform S3 upload/download, list buckets, DynamoDB insert/query, and run a Lambda function using Boto3
- Evaluate JFrog, Harbor, and Sonatype Nexus artifactories; select one and integrate with Jenkins
- Explore Istio service mesh and DaemonSet deployment on Kubernetes (Minikube)
- Research Grafana Loki architecture and Fluentd as a unified logging layer; compare the two
- Study Jaeger tracing with HotROD demo; explore running Jaeger on Kubernetes
- Research NRDB design principles and all types of testing (functional, integration, scale, performance, SLA monitoring)

### Problem Statement Clarification

---

**Scope Expansion:** Building on Week 1 foundations, the team now expands into observability, artifact management, and cloud-native tooling all of which feed directly into the multi-cloud performance evaluation pipeline.

#### Key Focus Areas for This Week:

- Observability: distributed tracing (Jaeger), log aggregation (Loki + Fluentd), and APM correlation
- Artifact management: understanding how container images and build artifacts are versioned and stored outside Docker Hub
- AWS emulation: proving all AWS service behaviour locally via LocalStack before any real cloud interaction
- Testing discipline: unit tests must be wired into the CI on merge pipeline no merge without a passing test gate
- Database design: understanding NRDB (New Relic Database) principles and test categories that map to SLA obligations

### Work Reviewed & Concepts Discussed

---

#### Observability Stack

- Jaeger distributed tracing: explored HotROD demo application as a practical introduction to trace propagation across services
  - Jaeger on Kubernetes: understood how to deploy Jaeger as a service inside a K8s cluster and expose it across multiple ports for UI, collector, and agent
-

- Jaeger is built on top of the OpenTelemetry standard; explored how React frontend can emit traces via browser-side OpenTelemetry SDKs
- AWS X-Ray: reviewed as the AWS-native tracing equivalent captures what happened inside the app across Lambda, API Gateway, and downstream services
- Grafana Loki: log aggregation system that stores logs and allows querying via LogQL; studied Loki architecture (distributor, ingester, querier, compactor)
- Fluentd: acts as a unified logging layer collects logs from all containers and forwards them to Loki or other backends, similar to CloudWatch agent on AWS
- Fluentd vs Loki: Fluentd is the collector/forwarder; Loki is the storage and query layer they work together, not as alternatives
- Humio: briefly reviewed as another log management platform (now part of CrowdStrike); noted as a commercial alternative to Loki
- NewRelic: reviewed NewRelic demo for full-stack APM covers traces, logs, metrics, and dashboards in a single SaaS platform

### **Docker Networking, Volumes & Scout**

- Docker networking modes reviewed: bridge (default, isolated network per container), host (shares host network stack), and overlay (multi-host swarm networking)
- Bridge networks: containers on the same bridge network communicate via container name DNS resolution; external traffic requires port mapping
- Node port mapping: mapping container ports to host ports using -p flag; understood the difference between EXPOSE in Dockerfile and actual port binding
- Docker Scout: security scanning tool built into Docker CLI; key commands include docker scout cves (list CVEs for an image) and docker scout recommendations (suggest fixes)
- Docker volumes: persistent storage that survives container restarts; bind mounts vs named volumes use cases and trade-offs discussed
- Docker Compose and Swarm reviewed: Compose for local multi-container orchestration; Swarm for simple production clustering

### **LocalStack & AWS Services via Boto3**

- LocalStack setup: runs all AWS services locally on port 4566; configured via Docker container with AWS CLI pointed to localhost endpoint
- awslocal CLI wrapper: drop-in replacement for aws CLI that auto-routes to LocalStack eliminates the need to pass -endpoint-url on every command
- S3 operations via Boto3: create bucket, upload file (put\_object), download file (get\_object), list buckets and objects all validated against LocalStack
- DynamoDB operations: create table with partition key, insert records (put\_item), query by key (get\_item / query), and scan explored single-table design principles
- Lambda on LocalStack: package a Python function, deploy via boto3, invoke it and inspect the response used to validate event-driven flow before AWS deployment
- Reference: AWS Boto3 docs and LocalStack integration guide used as the primary references for all SDK operations

---

## Artifact Repositories

- Compared three artifact repositories: JFrog Artifactory (commercial, feature-rich), Harbor (open-source, CNCF, focused on container images), and Sonatype Nexus (open-source, supports Maven, npm, Docker, PyPI, and raw formats)
- Nexus selected as the primary artifact repository due to open-source licensing and multi-format support; integrated with Jenkins as the downstream publish target after a successful build
- Artifact versioning: explored how artifacts are versioned per branch using semantic versioning and snapshot/release conventions prevents collision between feature branch and main builds
- Container image versioning: Harbor explored as a Docker Hub alternative for private image storage; supports vulnerability scanning and tag immutability
- AWS versioning via artifactory: discussed how S3 object versioning and ECR image tags relate to the same concept every artifact must be traceable back to a commit
- Role access and plugin management in Jenkins configured to control who can trigger pipelines and publish artifacts

## Unit Testing & CI Integration

- CI on merge flow: code pushed to feature branch → PR raised → unit tests run automatically → merge blocked if tests fail
- Test scope: unit tests cover individual component behaviour in isolation; integration tests cover interactions between components and services
- SonarQube security check integrated into the Jenkins pipeline as a quality gate pipeline fails if code coverage drops below threshold or critical issues are detected

## Database Design & Testing Principles

- NRDB (New Relic Database) design principles discussed: append-only, time-series optimised, high ingest throughput — contrasted with traditional RDBMS and document stores
- Test categories reviewed and mapped to the project:
  - Functional testing: does the feature behave as specified?
  - Integration testing: do services communicate correctly across boundaries?
  - Scale testing: does the system hold up under realistic concurrent load?
  - Performance testing: what are the latency and throughput characteristics?
  - SLA monitoring: are service level objectives (uptime, p99 latency) being met in production?

## Istio & Kubernetes Workload Types

- DaemonSet: ensures one pod runs on every node in the cluster used for log collectors (Fluentd), monitoring agents, and security scanners
  - StatefulSet: manages pods with stable network identities and persistent storage used for databases and message queues that require ordered, deterministic startup
  - Sticky sessions: configured via Istio DestinationRule using consistent hash load balancing routes a client to the same pod across requests based on header or cookie
  - Istio service mesh on Minikube: explored sidecar injection, traffic management with VirtualService and DestinationRule, and mTLS between services.
-

